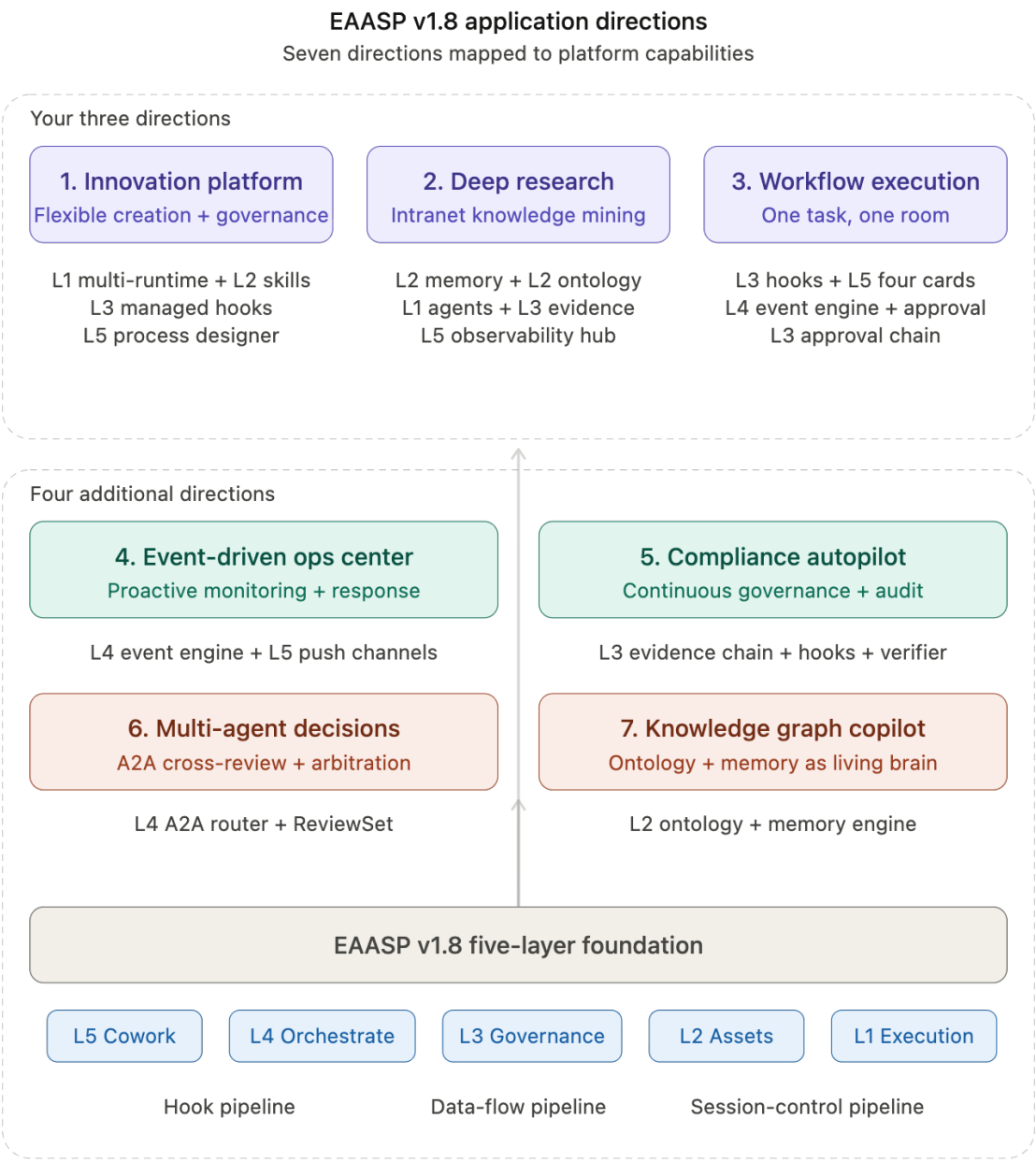
EAASP应用类型按“三类元范式“深度展开

Let me start by reading the uploaded document to understand EAASP v1.8's architecture and capabilities.Now let me search for relevant industry research on enterprise AI agent platform application patterns.Now I have a thorough understanding of both the EAASP v1.8 specification and the current enterprise AI agent landscape. Let me compose a comprehensive analysis.Now let me create a comprehensive visual analysis. Given the depth of the topic, I'll create an architectural diagram and then provide my detailed analysis.



After thorough reading of EAASP v1.8's 148K-word specification and cross-referencing with the current enterprise AI agent landscape, here's my analysis. Your three directions are well-conceived — they map cleanly onto EAASP's core architectural strengths. But the platform's design actually enables at least four more distinctive application directions that your initial framing hasn't yet covered.

Validating your three directions

Direction 1 — Innovation platform maps beautifully to EAASP's L1 multi-runtime architecture. The runtime selector's five strategies (task-matched, blind-box, user preference, A/B, cost-optimized) mean you can offer Claude Code-level creative freedom while L3 managed hooks provide the "deny always wins" guardrail. The process designer in L5 lets business users build workflow-skills without writing SKILL.md by hand. The key tension to manage: the 80/20 mode-router split (SIMPLE vs KERNEL) is critical here — routing everything through the kernel for "innovation" tasks will create unnecessary latency.

Direction 2 — Deep research leverages L2's Memory Engine (three-layer storage with evidence anchors, file-based memory, hybrid retrieval) and the Ontology Service. The evidence chain requirement — every conclusion must reference an anchor — is exactly what transforms casual AI summaries into professionally defensible business research. The platform's retrospective cycle (event → memory deposit → curated lessons) creates compounding research quality over time.

Direction 3 — Workflow execution is the most architecturally native direction. The "one task, one room" maps directly to L5's event room primitive (one room per business event). The "four cards pinned to the top" is literally EAASP's four-card paradigm (event card, evidence pack, action card, approval card). The five-stage approval chain (Plan → Check → Draft → Approve → Execute) gives you 100% policy execution through deterministic hooks rather than probabilistic prompts.

Four additional directions the architecture enables

Direction 4 — Event-driven operations center. This is perhaps the platform's most differentiated capability — and your three directions don't fully exploit it. L4's event engine transforms agents from passive responders into proactive scene-watchers. The spec is explicit: "one hundred alerts may constitute a single distribution-line overload event." The clustering pipeline (ingest → dedup → cluster → correlate → event object) with pluggable strategies (command/http/prompt/agent) means different industries can customize: spatial-topology clustering for power grids, transaction-pattern clustering for financial

services. Combined with L5's push channels (real-time SSE/WebSocket + async IM bots to WeChat Work, DingTalk, Feishu, Slack, Teams), this creates a proactive operations center where agents surface problems before humans even notice them. Deloitte's 2026 enterprise AI survey identifies supply chain management, R&D, knowledge management, and cybersecurity as particularly high-potential use cases for agentic AI, and this direction addresses all of those through event-driven monitoring.

Direction 5 — Compliance and audit autopilot. EAASP's governance layer is far richer than what's needed for simple workflow approval. The L3 evidence chain manager (validation, integrity verification, audit query support, compliance reporting), combined with the deterministic verifier (rule-engine, simulation, compliance, and numeric cross-validation), creates a continuous compliance engine. Every agent action generates telemetry → L3 audit service captures it → L5 observability hub surfaces it. The critical insight: L3's MCP security tiers (read-only, write, critical) with per-session credential issuance and minimum-privilege principle means you can build compliance into the fabric of work, not as an afterthought. Industry best practices emphasize that compliance should be embedded into system design from the beginning, with audit trails of all agent actions and decisions. EAASP's hook architecture makes this native.

Direction 6 — Multi-agent decision support. The A2A protocol (capability registration, collaboration dispatch, ReviewSet aggregation) enables a pattern far beyond simple task delegation. When L3 policy triggers A2A escalation — because of required cross-review, parallel evidence-gathering, trust-domain isolation, or the need for visible dissent — you get multi-dimensional analysis of complex decisions. The aggregation engine's three checks (conflict detection, confidence weighting, final recommendation) surface disagreement explicitly rather than hiding it behind a single "answer." This is transformative for risk-critical scenarios: investment decisions, safety reviews, legal assessments. Industry analysts report a 1,445% surge in multi-agent system inquiries from early 2024 to mid-2025, signaling massive demand for this pattern. EAASP's ReviewSet is ready-made for it.

Direction 7 — Enterprise knowledge graph copilot. L2's Ontology Service combined with the Memory Engine creates something that no other platform easily replicates: a living organizational brain. The ontology provides structured knowledge (equipment, personnel, processes, organizations, rules and their relationships), while the Memory Engine accumulates operational experience (preferences, thresholds, retrospective learnings). The feedback loop — execution data (L1) → audit telemetry (L3) → analytics (L5) → skill improvement (L2) → better execution — means the copilot gets smarter with every interaction. Agent-written memory starts as "agent_suggested" and requires human confirmation before promotion to "confirmed," creating a quality-controlled institutional knowledge base.

A deeper structural observation

Looking at the landscape, the seven directions actually fall into three meta-categories based on which EAASP vertical pipeline they primarily exercise:

The **hook pipeline** dominates Directions 3 (workflow execution) and 5 (compliance autopilot) — these are about deterministic guarantees.

The **data-flow pipeline** dominates Directions 2 (deep research), 4 (event-driven ops), and 7 (knowledge copilot) — these are about intelligence flowing through the system.

The **session-control pipeline** dominates Directions 1 (innovation platform) and 6 (multi-agent decisions) — these are about orchestrating complex, multi-session collaborative work.

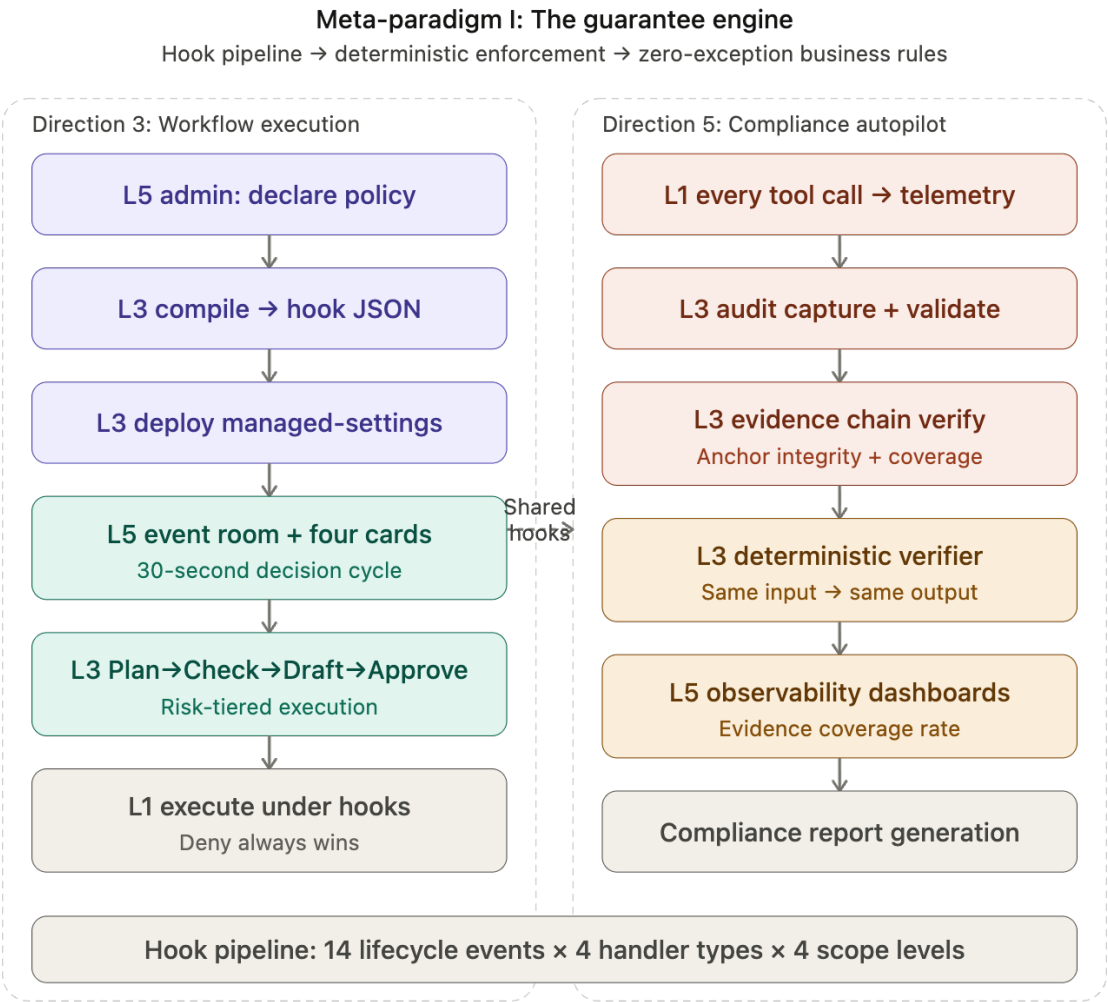
This mapping suggests a useful prioritization heuristic: deploy each pipeline's "primary" application first to validate the infrastructure, then expand to secondary applications on the same pipeline with minimal additional investment.

Gartner projects that 40% of enterprise applications will embed AI agents by end of 2026, and agentic AI lock-in accumulates at multiple layers — the foundation model, orchestration framework, runtime environment, and developer patterns. EAASP's runtime-agnostic L1 with its 12-method interface contract directly mitigates this multi-layer lock-in risk, which is a significant strategic advantage for all seven directions.

Want me to dive deeper into any specific direction — for instance, mapping it to concrete skill designs, hook configurations, or the evolution-phase timeline?

Expand in depth by the three meta-paradigms.

Let me re-read the key architectural sections to ensure precision on the three vertical pipelines before expanding.Now let me build the three deep-dive visualizations and analysis.



Meta-paradigm I: The guarantee engine

The hook pipeline is EAASP's most architecturally distinctive invention. It threads deterministic enforcement through all five layers — from administrator intent in L5, through policy compilation in L3, to inline execution interception in L1. The two application directions that primarily exercise this pipeline share a common design philosophy: **things that must happen every time are not suggestions to the agent; they are deterministic rules enforced by code**.

The architectural spine is straightforward: administrators declare business-language policies in L5's admin console → L3's policy compiler translates them into executable hook configurations (choosing among four handler types: command at millisecond latency, HTTP at ~300ms, prompt at ~2s, agent at ~15s) → deployment is atomic via managed-settings.json → L1's hook bridge ensures enforcement across all runtimes regardless of whether they natively support hooks. The 14 lifecycle events across three layers (9 in L1, 2 in L3, 3 in L4) provide governance coverage of the complete request lifecycle from event intake to memory deposit.

Direction 3 (Workflow execution) exploits the hook pipeline's *prescriptive* face — hooks that shape what work looks like. The "one task, one room" model maps directly to L5's event room primitive: when an event arrives (whether user-initiated or system-triggered), an event room is automatically created, and the four pinned cards structure the employee's cognitive process. The design spec is explicit that each card must enable a decision within 30 seconds. This is where the five-stage approval chain becomes critical: for low-risk operations (read-only queries, summaries), the agent completes autonomously — all five stages are skipped. For medium-risk operations (report generation, suggested modifications), execution proceeds to Draft and pushes a notification without forcing approval. For high-risk operations (config changes, work orders, equipment operations), the full Plan → Check → Draft → Approve → Execute chain activates.

The skill design pattern here is the **workflow-skill**: a SKILL.md file whose YAML frontmatter carries scoped hooks that activate only during that skill's execution and automatically clean up when it finishes. The prose instructions tell the agent *what good looks like* (quality criteria, edge cases, adaptation strategies), while the frontmatter hooks *guarantee* that certain checks happen (PreToolUse for domain safety, PostToolUse for quality evaluation, Stop for completeness validation). Intelligence and guarantee coexist in a single asset.

Concrete skill examples for workflow execution:

- **Procurement approval workflow**: PreToolUse command hook blocks any purchase order tool call exceeding the employee's spending authority. PostToolUse prompt hook evaluates whether the vendor selection rationale references at least two comparison bids. Stop hook verifies all required fields are populated before the document enters the approval card.
- **Equipment maintenance dispatch**: The deterministic verifier runs a simulation check (type: "feasibility_check") against the maintenance plan before it enters the Draft stage. If the simulation finds a scheduling conflict or safety violation, the plan is blocked — this is not an LLM judgment call, it's a rule engine producing the same output for the same input every time.
- **Monthly close process**: A workflow-skill with 12 sequential steps, each with its own PostToolUse prompt hook evaluating output quality. The Stop hook verifies all 12 steps produced outputs and that the evidence coverage rate (proportion of conclusions with attached evidence anchors) meets the department's threshold.

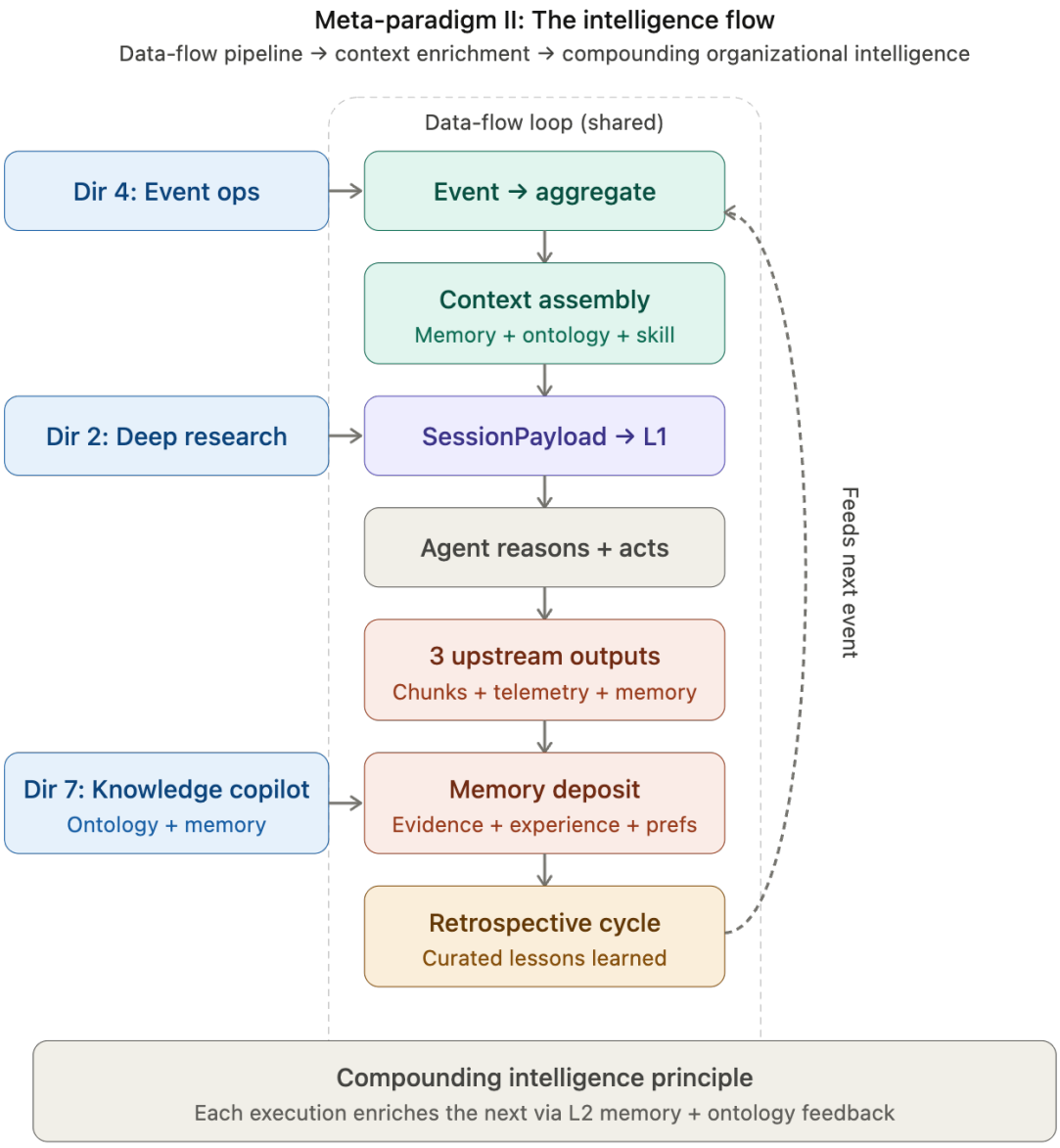
Direction 5 (Compliance autopilot) exploits the hook pipeline's *observational* face — hooks that capture, validate, and report. Every tool call in every L1 session generates telemetry through async PostToolUse HTTP hooks: session ID, runtime ID, user identity, tool name, sanitized inputs/outputs, execution duration, hook decisions, and resource consumption. This telemetry flows L1 → L3 audit service → L4 observability hub → L5 dashboards, creating a continuous compliance pipeline.

The evidence chain manager in L3 adds a layer beyond simple audit logging. It validates that agent outputs attach required evidence anchors, confirms that referenced data snapshots exist and are intact, enables traceback from any conclusion to its complete evidence chain, and generates evidence-chain-organized audit reports. The design spec's principle — "every agent conclusion must reference at least one evidence anchor" — transforms compliance from periodic review into continuous, automatic enforcement.

Concrete application patterns for compliance autopilot:

- **Real-time policy drift detection**: A PostToolUse prompt hook evaluates every agent output against the organizational policy for that department. If the output contains language or recommendations that contradict standing policy, the hook blocks with a structured reason, and the block event enters the compliance dashboard with full context.
- **Evidence completeness monitor**: The L5 observability hub tracks the evidence coverage rate — the proportion of agent conclusions that have attached evidence anchors. When coverage drops below a configurable threshold (say, 95%), a budget alert fires. This catches the subtle failure mode where agents produce reasonable-sounding conclusions without grounding them in data.
- **MCP security tier enforcement**: L3's three-tier MCP classification (read-only, write, critical) with per-session credential issuance creates automatic compliance with the minimum-privilege principle. When a financial analyst's agent tries to invoke a write tool on a production database during business hours, the PreToolUse command hook blocks instantly — millisecond latency, zero ambiguity.
- **Regulatory audit package generation**: When an external auditor requests evidence for a decision, the evidence chain manager can trace from the final conclusion back through every intermediate step to the original data snapshots. Each anchor contains the source system, tool version, model version, rule version, and timestamps. This isn't "what the AI said it did" — it's a cryptographically verifiable chain of what actually happened.

The key tension in this meta-paradigm is the cost-latency escalation ladder. Command hooks cost milliseconds and nothing. Prompt hooks cost 1-3 seconds and LLM inference tokens. Agent hooks cost 5-30 seconds and significant compute. The design anti-pattern the spec warns against — "overusing expensive hooks" — is especially dangerous in the guarantee engine paradigm because guarantee applications tend to want comprehensive checking. The discipline is: start with command hooks for deterministic rules (file path checks, deny-list matching, format validation), escalate to prompt hooks only when semantic evaluation is genuinely needed (quality assessment, anomaly detection), and reserve agent hooks for complex validation that requires multi-step reasoning (numerical reconciliation, comprehensive code testing).



Meta-paradigm II: The intelligence flow

The data-flow pipeline is where EAASP becomes more than a governance framework — it becomes a *learning organization platform*. The pipeline has two directions: downstream (raw events and user requests are enriched with context, memory, and skill instructions as they descend from L5 through L4/L3/L2 into L1) and upstream (agent execution produces three output types — response chunks for streaming, telemetry events for audit, and memory writes for persistent knowledge — that flow back up through the layers). The crucial architectural innovation is the **closed loop**: the retrospective cycle at the end of every event deposits curated memory back into L2, where it enriches the context assembly for the *next* event. Every execution makes the platform smarter.

Three application directions primarily exercise this pipeline, each attaching at a different point in the loop.

Direction 4 (Event-driven operations center) attaches at the *top* of the downstream flow — the event engine. This is the most underappreciated capability in your initial three-direction framing. The event engine's processing pipeline (ingest → dedup → cluster → correlate → event object) is designed as pluggable strategies supporting the same four handler types as hooks. This means different industries customize clustering logic: a power-grid operator uses spatial-topology-aware clustering that understands which alerts come from equipment on the same distribution line; a financial institution uses transaction-pattern clustering that recognizes when dozens of small anomalies indicate a single systematic issue; a hospital system uses patient-journey-aware clustering that connects scattered alerts into a coherent clinical picture.

The event state machine (received → triaging → active → pending_approval → executing → monitoring → closed → retrospective) provides the operational backbone. The EventReceived hook fires on every event intake, enabling policy-based filtering before the event even enters the pipeline — critical for preventing alert fatigue. The design spec is explicit: "one hundred related alerts should aggregate into a single event object, not spawn one hundred event rooms."

Concrete ops center patterns:

- **Proactive incident detection**: The event engine correlates a monitoring alert (server CPU spike), a ticketing system event (customer complaint), and a CDC trigger (database connection pool exhaustion) into a single "service degradation event." L5 pushes a notification to the on-call engineer's DingTalk/Slack with a pre-assembled event card showing severity, impact scope, and minimum viable next action — before anyone has filed a ticket.

- **Adaptive collaboration tempo**: Simple events (a resolved alert that just needs acknowledgment) → the agent completes autonomously and pushes a result card. Judgment events (multiple possible root causes) → the agent produces an action card requesting human input. Sensitive events (production config changes required) → the full approval chain activates. The tempo is automatically determined by L3's policy engine, not by the agent's own assessment.
- **Cross-source event correlation**: The connector fabric (ETL/ELT between enterprise systems and L4) feeds organizational structure, employee data, and business metrics into L4's databases. When the event engine correlates a monitoring alert with a recent deployment recorded in the change management system, the resulting event card already includes the deployment diff — the agent doesn't need to go searching for it.

Direction 2 (Deep research) attaches at the *middle* of the downstream flow — context assembly. When L4's orchestrator creates a SessionPayload for a research task, it queries L2's Memory Engine for relevant prior memory (using hybrid retrieval: keyword + semantic + time-decay) and L2's Ontology Service for business context (ontology_query for type/attribute/relation queries, ontology_traverse for relationship-path traversal, ontology_context for context packs around specified objects). The context assembly priority is explicit: managed policy context (highest) → event context → relevant memory → skill instructions → user preferences (lowest). When the context window is constrained, lower-priority content is trimmed first — which means policy and evidence always survive, even if some historical memory gets compressed.

The evidence anchor requirement transforms research from "the AI said X" into "the AI concluded X based on evidence anchors Y_1, Y_2, Y_3, each referencing an immutable data snapshot with integrity hash." This is the difference between an AI-generated summary and a professionally defensible research deliverable.

Concrete research application patterns:

- **Individual business report (人物调研报告)**: A research session pulls from the Ontology Service (organizational relationships, reporting structure, project assignments), the Memory Engine (prior interactions, historical assessments, noted preferences), and external MCP connectors (email patterns via Gmail MCP, calendar commitments via Calendar MCP, document authorship via Drive MCP). Each claim in the report references evidence anchors. The Stop hook verifies completeness: all required sections present, evidence coverage above threshold, no unsubstantiated claims.

- **Routine briefing materials (例行简报)**: An event-driven pattern. A cron trigger (scheduled task) fires daily at 07:00 → the event engine creates a "morning briefing" event → L4 assembles context from overnight monitoring events, external news via web search MCP, and internal metrics from the connector fabric → the agent produces a briefing document with four sections (overnight incidents, metrics summary, today's agenda items from Calendar MCP, action items carried forward from Memory Engine). Each iteration deposits its briefing conclusions into L2 memory, so the agent accumulates understanding of what the reader cares about over time.
- **Archive-based compilation and analysis (档案编研)**: The Memory Engine's three-layer storage is purpose-built for this. Evidence anchors (Layer 1) provide immutable references to original documents. File-based memory (Layer 2) accumulates compiled knowledge in human-reviewable structured text. Hybrid retrieval (Layer 3) enables both exact-match queries ("find all documents referencing Project Phoenix") and fuzzy similarity queries ("find events similar to the 2024 Q3 supply chain disruption"). The cross-agent sharing rules mean that a compilation project can span multiple research sessions — each session deposits findings into memory, and the next session picks up where the last left off.
- **Training courseware (培训课件)**: A multi-session workflow. Session 1 (research agent) gathers domain knowledge and deposits it into L2 memory. Session 2 (content authoring agent, possibly on a different runtime with production-skill for PPTX output) pulls the research memory and produces courseware. Session 3 (quality evaluation agent) reviews against the training standards domain-skill. The sessions pass context through L2 Memory, not conversation history — which means each session can use the optimal runtime for its task.

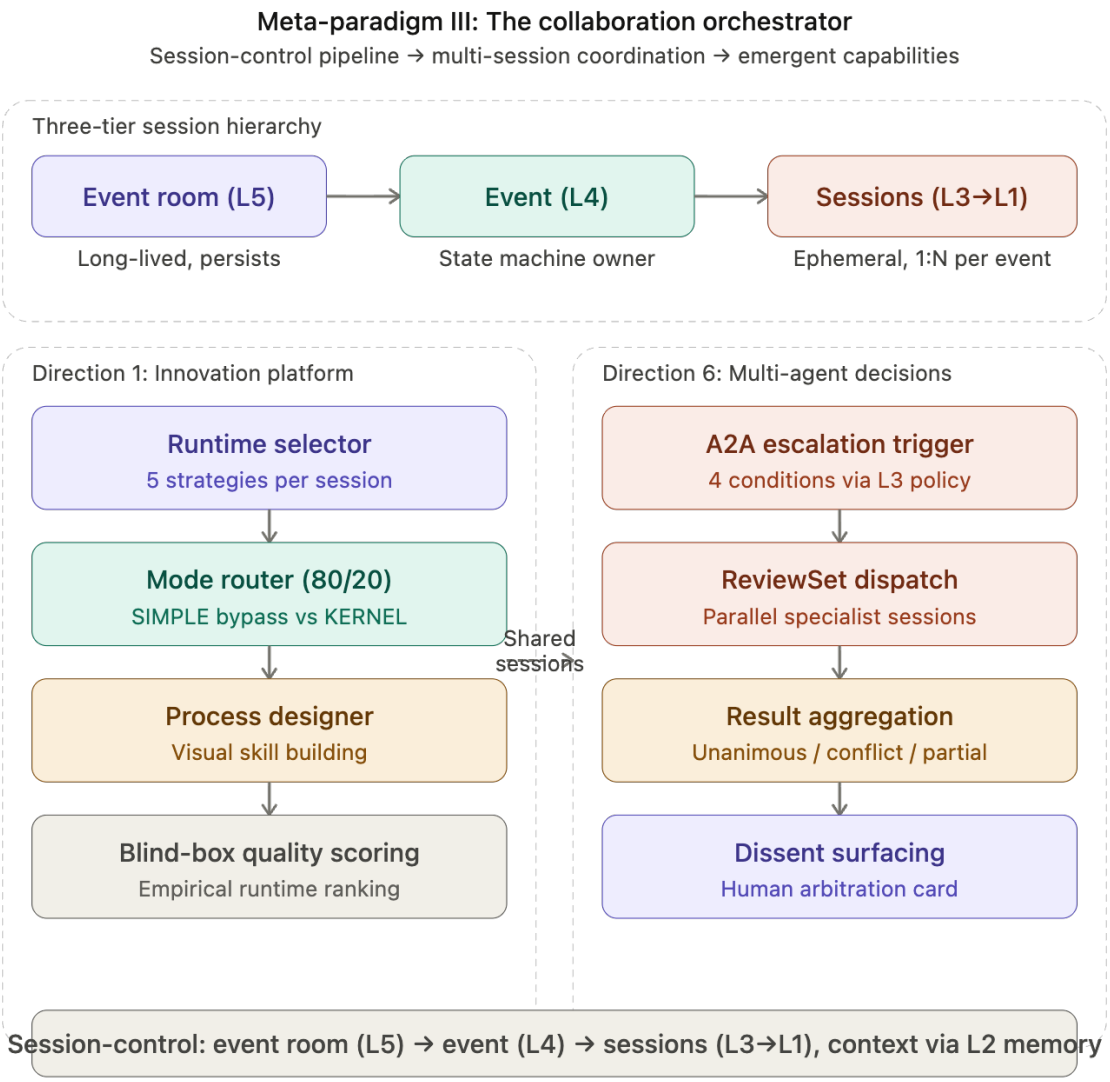
Direction 7 (Knowledge graph copilot) attaches at the *bottom* of the upstream flow — memory deposit and retrospective. This is the most strategically valuable long-term direction because it creates compounding returns. The Ontology Service provides the structural skeleton (what entities exist and how they relate), while the Memory Engine accumulates operational flesh (what happened to those entities, what preferences emerged, what calibrations proved correct).

The memory lifecycle is carefully governed: agent-written memory starts as "agent_suggested" → human confirmation promotes it to "confirmed" → eventual archival (soft-delete with audit trail). This prevents agent drift — if the agent develops incorrect assumptions about business rules, a human can correct the memory file and the correction propagates to all future sessions.

Concrete knowledge copilot patterns:

- **Living organizational brain**: When an employee asks "who handled the last procurement issue similar to this one?", the copilot queries ontology_traverse (organizational relationships) + memory_search (keyword: "procurement", category: "experience", scope: "department", time-decay weighted toward recent events) → returns not just a name but the complete experience chain: what worked, what didn't, what the agent learned about that specific vendor from the retrospective cycle.
- **Threshold calibration**: Over time, agents learn operational thresholds — "this server typically runs at 60% CPU; an alarm at 70% is probably a false positive, but 85% correlates with real incidents." These calibrations are deposited as calibration-category memory files. New agents on the same team inherit these thresholds via context assembly, avoiding the cold-start problem.
- **Cross-domain knowledge synthesis**: The ontology_context tool returns context packs around specified objects. When an event affects a piece of equipment, the context pack includes not just the equipment's attributes but its relationships to personnel, processes, and organizational units. Combined with memory_search across all those related entities, the copilot can surface insights that no single department's knowledge would reveal — "this equipment failure pattern has occurred three times in the past year, each time after a specific supplier's maintenance window."

The compounding intelligence principle that unites all three directions: each execution enriches the memory base → each enrichment improves context assembly for the next execution → improved context produces better conclusions → better conclusions deposit richer memory. The retrospective cycle (the final stage of the event state machine) is where this compounding is made explicit: a dedicated session reviews the complete event history and produces curated memory files summarizing lessons learned, threshold calibrations, and operational experience. Unlike simple conversation history accumulation, this is deliberate, governed, human-reviewable knowledge curation.



Meta-paradigm III: The collaboration orchestrator

The session-control pipeline manages the most complex relationship in EAASP: the three-tier lifecycle hierarchy. An Event Room in L5 is long-lived — created when the event is created, persists across all sessions, archives only after the retrospective phase. An Event in L4 owns the authoritative state machine (received through retrospective). Sessions in L3→L1 are ephemeral — one event can spawn multiple sessions for initial analysis, follow-up investigation, A2A parallel reviews, and post-approval execution. The critical design decision: **sessions pass context through L2 Memory, not conversation history**. When Session 1 ends, its conclusions are deposited into L2. When Session 2 starts, the orchestrator pulls relevant memory into the new SessionPayload. This is more robust than raw conversation forwarding because memory is curated, versioned, and access-controlled.

The two application directions that primarily exercise this pipeline share a common need: **orchestrating multiple agents and sessions into coherent, higher-order outcomes that no single session could produce alone**.

Direction 1 (Innovation platform) is the most architecturally demanding direction because it requires the full breadth of the session-control pipeline while maintaining the creative freedom that makes innovation possible. The fundamental tension is between L3's "deny always wins" governance and the open-ended exploration that innovation demands. EAASP resolves this elegantly through three mechanisms:

First, the runtime selector. The five selection strategies (task-matched, random blind-box, user preference, A/B testing, cost-optimized) let the innovation platform offer genuine diversity of capability. A coding innovation task routes to Claude Code (strongest coding capability, powered by Claude Opus 4.6). A document-heavy innovation task routes to Octo (enterprise personal assistant focus with kernel boot skill and mode router). A data-pipeline innovation routes to a LangGraph agent (graph-based orchestration with durable execution and crash recovery). The user can also explicitly choose (user preference strategy), constrained only by policy — administrators can restrict certain runtimes for certain roles.

Second, the mode router. The spec warns against routing all requests through the kernel — 80% of requests should be classified as SIMPLE with zero kernel overhead. For an innovation platform, this is essential: when an employee is in rapid creative iteration (brainstorming, quick prototyping, exploring alternatives), SIMPLE mode provides immediate responsiveness. When the exploration crystallizes into a concrete plan that touches production systems, the mode router shifts to KERNEL, activating the structured execution protocol (intent parser, resource coordinator, execution loop) with full governance.

Third, the process designer. L5's visual, form-based tool for business analysts to create workflow-skills without writing SKILL.md by hand is the bridge between innovation and institutionalization. When an employee discovers a valuable workflow through innovation, they can formalize it through the process designer, which compiles their input into SKILL.md format and submits to the skill repository via the skill lifecycle API. The simulation mode (sandboxed L1 session with read-only MCP connectors) lets them test before committing. This creates a **innovation** → **formalization** → **governance** pipeline: creative discovery in the innovation platform → workflow-skill creation in the process designer → promotion pipeline (draft → tested → reviewed → production) → governed execution in the workflow execution direction.

The blind-box quality mechanism is unique to the innovation platform. The random blind-box selection strategy assigns two runtimes simultaneously, presents both outputs, and the user votes up/down without knowing which runtime produced which result. This generates pairwise comparison data for empirically grounded quality scoring — not self-reported preferences, but blind evaluations of actual output quality. Over time, the runtime comparison analytics in L5's observability hub build a quality heat map: which runtimes excel at which task types, which have the lowest failure rates, which produce the highest user satisfaction. This data feeds back into the task-matched strategy, continuously improving runtime selection.

Concrete innovation platform patterns:

- **Citizen development with guardrails**: A business analyst uses the process designer to build a customer-churn analysis workflow-skill. During development, they iterate freely in SIMPLE mode with Claude Code runtime. The skill's frontmatter hooks enforce data handling policies (PreToolUse command hook blocks access to PII columns; PostToolUse prompt hook evaluates whether the analysis methodology is statistically sound). The analyst never had to think about governance — it's embedded in the platform.
- **Multi-runtime creative pipeline**: An innovation project starts with research (Octo runtime, pulling from Memory Engine and Ontology Service), moves to prototyping (Claude Code runtime, building a proof-of-concept application), then to documentation (Octo runtime with production-skill for DOCX output). Each phase is a separate session; context passes through L2 Memory. The innovation platform manages this multi-runtime pipeline through L4's session orchestrator, with intent changes creating new sessions automatically.
- **Runtime competition as innovation engine**: Department heads enable blind-box testing for 10% of innovation sessions. Over a quarter, the platform accumulates enough quality data to determine that for their specific workloads, Aider outperforms Claude Code on pure refactoring tasks while Claude Code dominates on greenfield architecture. This empirical data — not vendor benchmarks — drives runtime allocation policy.

Direction 6 (Multi-agent decision support) exploits the most sophisticated feature of the session-control pipeline: A2A coordination via the ReviewSet mechanism. The default is single-agent operation — A2A escalation occurs only when four specific conditions are met (per L3 policy): specialized cross-review is needed, parallel evidence-gathering reduces latency, different trust domains require isolation, or humans need to see dissent rather than a single conclusion.

The ReviewSet is the core orchestration unit. When L4's A2A router decides to escalate, it creates a ReviewSet containing the set of parallel sessions, each bound to a different specialist agent (a Skill + Runtime combination with registered capability descriptors). Each session receives the shared event context and independently produces a conclusion with evidence anchor IDs and a confidence score. The aggregation engine then applies three checks:

Conflict detection compares conclusions across sessions. If Session A's protection analysis says "safe to proceed" while Session B's compliance review says "regulatory risk identified," the conflict is flagged with specific contradiction details — not buried in a merged summary.

Confidence weighting weights each conclusion by the agent's historical quality score (from blind-box voting and usage analytics). An agent with a 0.92 quality score on protection analysis carries more weight than one with a 0.74.

Final recommendation merges into one of three outcomes: unanimous (merge into unified action card), conflicting (flag dissent and require human arbitration via the approval card), or partial (completed portions presented with pending items noted). The PreResultMerge and PostResultMerge hooks fire during aggregation, enabling L3 governance to validate completeness and consistency.

Concrete multi-agent decision patterns:

- **Investment committee decision support**: A significant capital expenditure proposal triggers A2A escalation (high-risk threshold in L3 policy). Three parallel sessions: a financial analysis agent (LangGraph runtime with access to ERP MCP), a regulatory compliance agent (Octo runtime with legal domain-skill), and a market risk agent (Claude Code runtime with web search for current market data). Each produces an independent assessment with evidence anchors. The ReviewSet's aggregation surfaces any disagreements explicitly — if the financial analysis says "strong ROI" but the market risk agent flags "sector headwinds," both positions appear in the action card, and the approval card requires human arbitration.
- **Safety incident investigation**: A safety event triggers A2A escalation because multiple trust domains are involved. An equipment-specialist agent (with access to the equipment maintenance MCP and the ontology for equipment relationships) runs in parallel with a process-safety agent (with access to the safety procedures domain-skill and historical incident memory) and a regulatory agent (with read-only access to the regulatory database MCP). Trust domain isolation means each agent can only access the MCP connectors relevant to its specialty — the equipment agent cannot see HR data, and the process-safety agent cannot modify equipment configurations.

- **Cross-functional strategic review**: A quarterly strategic initiative requires input from multiple departments. Rather than sequential meetings, the A2A router dispatches parallel sessions to agents configured for each department's perspective (sales forecasting, operations capacity, finance modeling, HR talent planning). Each session pulls from department-scoped memory in L2. The aggregation identifies where departmental priorities conflict and surfaces these conflicts for executive arbitration — transforming weeks of alignment meetings into a structured decision package.

The emergent capability of this meta-paradigm is that collaboration orchestration enables a form of organizational intelligence that doesn't reside in any single agent or any single human. The innovation platform generates new workflow-skills that become shared organizational assets. Multi-agent decisions produce conclusions that incorporate more perspectives than any individual could hold simultaneously. The session-control pipeline's three-tier hierarchy (long-lived rooms → state-machine events → ephemeral sessions) provides the temporal structure for this to unfold naturally — rooms persist as organizational memory, events capture the decision logic, sessions do the work and then disappear.

Now let me pull together the cross-cutting synthesis.

Core build · Extension · Advanced

Direction	Ph 0 MVP	Ph 1 Events	Ph 2 Memory	Ph 3 Approval	Ph 4 Multi-agent	Ph 5 Collab	Ph 6 Ecosystem
I · Guarantee engine							
3. Workflow exec	Hook chain	Event rooms	—	Full 5-stage	—	Four cards	Marketplace
5. Compliance	Audit hooks	Event filter	Evidence chain	Verifier	—	Dashboards	Reg. packs
II · Intelligence flow							
4. Event ops	—	Engine core	Context enrich	Risk tempo	—	Push + IM	Cross-source
2. Deep research	Basic query	—	Memory + ontol	—	—	Brief gen	Archive AI
7. Knowledge copilot	—	—	Memory engine	—	—	Retro cycle	Living brain
III · Collaboration orchestrator							
1. Innovation	Single runtime	—	—	—	Blind-box	Designer	Multi-RT
6. Multi-agent	—	—	Shared mem	Review gate	A2A + ReviewSet	Dissent card	Ecosystem

Each cell shows what the direction gains at that phase. Core build = primary dependency. Extension = enhanced capability. Advanced = full potential.

Cross-paradigm synthesis: deployment strategy and mutual reinforcement

The deployment matrix above reveals something important: the seven directions are not independent product bets — they share infrastructure in a way that creates compounding returns across evolution phases.

Phase 0 (MVP) delivers three directions simultaneously. The basic hook chain, audit hooks, and single-runtime execution that Phase 0 validates are sufficient for minimal viable versions of Direction 3 (workflow execution with hooks but no approval chain yet), Direction 5 (compliance with audit capture but no evidence chain yet), and Direction 1 (innovation on a single runtime). This is the highest-leverage starting point: one phase of infrastructure work, three immediately usable application surfaces.

Phase 2 (Memory and evidence) is the inflection point. Weeks 5-12 deliver the Memory Engine and evidence anchors, which simultaneously unlock Direction 2 (deep research becomes possible when context assembly can pull from persistent memory and ontology), Direction 7 (knowledge copilot becomes possible when the memory deposit cycle exists), and dramatically enhance Direction 5 (compliance goes from "recording what happened" to "proving why this judgment was made" once evidence chains are operational). This phase should receive disproportionate engineering investment because it enables three directions and enhances a fourth.

The mutual reinforcement pattern across paradigms is the deepest strategic insight:

The guarantee engine (Paradigm I) produces telemetry and evidence chains → which feed the intelligence flow (Paradigm II) with structured operational data → which builds organizational memory that makes the collaboration orchestrator (Paradigm III) smarter when assembling context for multi-session and multi-agent work → which produces richer conclusions that deposit back into memory → which creates better evidence for compliance auditing.

This is not a sequential dependency — it's a reinforcing cycle. Each paradigm's outputs are another paradigm's inputs. The more workflows you execute under governance (Paradigm I), the richer your knowledge base becomes (Paradigm II), the better your multi-agent decisions get (Paradigm III), which produces higher-quality evidence (back to Paradigm I).

The practical implication for product planning: don't think of these as seven separate products with seven separate roadmaps. Think of them as seven *surfaces* on a single platform engine, each surface exposing a different combination of the three vertical pipelines. The engineering team builds pipelines; the product team composes surfaces. A single new platform capability (say, the A2A router in Phase 4) simultaneously enables

Direction 6 (multi-agent decisions as its primary surface) and enhances Direction 2 (deep research can now use parallel specialist agents for multi-dimensional analysis) and Direction 4 (event-driven ops can now dispatch multi-agent investigation for complex incidents).

This "pipeline infrastructure, surface composition" model is how you extract maximum application breadth from the 10-16 person team the spec describes in Chapter 21 — the engineers build shared infrastructure once, and each application direction is a composition of that shared infrastructure with direction-specific skills, hooks, and L5 rendering configurations.